PROJETO IMPLEMENTAÇÃO AES-128 EM DOIS ESP-32 VIA UART

Disciplina: Segurança de Sistemas de computação

Equipe: Lorena Strobel Campos (2023278440028) & Mateus de Souza Arruda (2023278440030)

Repositório: SSC-Criptografia-Simetrica (*GitHub*)

1. INTRODUÇÃO

Este relatório tem o objetivo de documentar a implementação do algoritmo criptográfico AES-128 sem o uso de bibliotecas de criptografia, operando no modo *Electronic Codebook* (ECB). O sistema foi dividido em dois módulos executados em microcontroladores ESP32 comunicando-se via UART2 (115200 bps, 8N1). A Placa A (Transmissor) é responsável pela derivação de chaves, preenchimento PKCS#7 e cifragem da mensagem, enquanto a Placa B (Receptor) atua na sincronização de pacotes brutos, decifragem inversa e remoção do preenchimento.

2. GERAÇÃO DA CHAVE E MENSAGEM A SER RECEBIDA

O algoritmo utiliza uma chave simétrica de 16 bytes derivada das matrículas da dupla (Lorena: `2023278440028` e Mateus: `2023278440030`). Conforme as especificações, foram extraídos os 8 últimos dígitos de cada uma e depois convertidos para o padrão ASCII.

- **Chave (ASCII):** `7844002878440030`
- **Chave (Hexadecimal):** `37 38 34 34 30 30 32 38 37 38 34 34 30 30 33 30`

A mensagem em texto claro a ser transmitida engloba as matrículas completas no formato: **`A:<matrícula A>;B:<matrícula B>`.**

- **Mensagem Base:** **`A:2023278440028;B:2023278440030`** (31 bytes).
- **Preenchimento (PKCS#7):** Para que a mensagem se torne um múltiplo exato de 16 bytes (128 bits), foi adicionado 1 byte de preenchimento com valor `0x01`, conforme a regra `16 - (31 mod 16) = 1`.
- **Mensagem com PKCS#7 (Hexadecimal):**
  - *Bloco 1:* **`41 3a 32 30 32 33 32 37 38 34 34 30 30 32 38 3b`**
  - *Bloco 2:* **`42 3a 32 30 32 33 32 37 38 34 34 30 30 33 30 01`**
- **Chave de Rodada 0 (Hexadecimal):** `37 38 34 34 30 30 32 38 37 38 34 34 30 30 33 30`
- **Chave de Rodada 1 (Hexadecimal):** `32 fb 30 30 02 cb 02 08 35 f3 36 3c 05 c3 05 0c`

3. ANÁLISE DO PROCESSO DE CIFRAGEM E EFEITO AVALANCHE (PLACA A)

3.1. Cinco Estados do Bloco 1 (Rodada 1)

Registros das transformações dos 16 primeiros bytes da mensagem (Bloco 1) ao longo da primeira rodada:

1. **`AddRoundKey` Inicial (Chave 0 - Whitening):**
   `76 02 06 04 02 03 00 0f 0f 0c 00 04 00 02 0b 0b`
2. **`SubBytes`:**
   `38 77 6f f2 77 7b 63 76 76 fe 63 f2 63 77 2b 2b`
3. **`ShiftRows`:**
   `38 7b 63 2b 77 fe 2b f2 76 77 6f 76 63 77 63 f2`
4. **`MixColumns`:**
   `b5 40 f8 06 2e 1f d2 b3 6c 5f 45 6e ce da df 4e`
5. **`AddRoundKey` (Chave 1):**
   `87 bb c8 36 2c d4 d0 bb 59 ac 73 52 cb 19 da 42`

- **Texto Cifrado Final (Todos os blocos):**
  `20 19 19 F9 9F 96 20 2A EF 26 CD FF 9E B3 37 4D 5C 7F A1 6D 88 CB 3C 29 F0 AE C6 B3 C2 04 D9 66`

3.2. Efeito Avalanche

O teste do efeito avalanche foi conduzido alterando-se exatamente 1 bit da chave simétrica mestre (inversão do bit menos significativo do primeiro byte: `'7' = 0x37 0x01 = 0x36 -> '6'` $\oplus$, gerando a chave "`6844002878440030`" via comando interativo `/flip` no Transmissor):

- **Texto cifrado original (Bloco 1):**
  `20 19 19 F9 9F 96 20 2A EF 26 CD FF 9E B3 37 4D`
- **Texto cifrado após alteração de 1 bit:**
  `E5 49 E0 66 90 D1 0F 1D 71 32 41 7A 5B 7E 06 29`
- **Análise Quantitativa:** A comparação bit a bit (distância de Hamming) entre os dois blocos cifrados revelou **64 bits modificados de um total de 128 bits**, correspondendo a uma variação de **exatos 50,00%**. Esse resultado comprova o cumprimento pleno do Critério de Avalanche Estrito (*Strict Avalanche Criterion* - SAC), demonstrando a eficácia combinada da substituição não linear da **S-Box** com a difusão linear das camadas `ShiftRows` e `MixColumns`.

4. ARQUITETURA E PROCESSAMENTO DO RECEPTOR (RX — Placa B)

Para garantir a correta sincronização dos dados cifrados pela UART2, a máquina de estados da Placa B foi programada para aguardar um byte fixo de sincronismo (`0xAA`). Após o alinhamento, o sistema lê dinamicamente a quantidade de blocos (n) informada e aguarda a recepção de exatos $16 \times n$ bytes brutos. Foi implementada uma janela de tempo limite (`timeout`) de 2000 milissegundos; caso o quadro de transmissão sofra interrupções e o tamanho declarado não seja recebido, o pacote é descartado para prevenir travamentos no loop de escuta.

A decifragem atua sobre os blocos independentes aplicando as funções inversas do AES: `AddRoundKey` (com a subchave da rodada 10), seguido por 9 iterações de `InvShiftRows`, `InvSubBytes`, `AddRoundKey` e `InvMixColumns`, e uma rodada final sem a mistura de colunas. Por fim, o algoritmo varre o último bloco validando se os bytes de preenchimento coincidem com a regra matemática do PKCS#7, removendo-os do buffer e inserindo o terminador nulo (`\0`) para a recuperação exata do texto original

5. DISCUSSÃO SOBRE O MODO ECB

Para estender a aplicação de uma cifra de bloco a mensagens com comprimento superior a 16 bytes, como no caso da mensagem composta pelas matrículas, que totaliza 31 bytes e, após o preenchimento PKCS#7, alcança 32 bytes (equivalente a 2 blocos AES), é indispensável o uso de um modo de operação criptográfico. No projeto, foi utilizado o **ECB (*Electronic Codebook*)**, abordagem na qual o texto claro preenchido é dividido em segmentos contíguos de tamanho fixo (16 bytes), aplicando-se a transformação AES-128 de maneira totalmente isolada e independente em cada um deles.

6. POR QUE O MODO ECB NÃO DEVE SER UTILIZADO

O modo ECB apresenta graves vulnerabilidades, razão pela qual seu uso é desaconselhado nas aplicações práticas modernas de segurança. O seu principal problema estrutural é a **ausência de difusão inter-blocos e o determinismo**: como não há encadeamento nem uso de um Vetor de Inicialização (IV), blocos de texto claro idênticos produzem invariavelmente blocos cifrados idênticos sob a mesma chave.

Em um cenário real de comunicação em barramento, um invasor que monitore o tráfego consegue identificar facilmente a repetição de padrões e a periodicidade das transmissões, sem precisar quebrar a chave criptográfica. Além disso, pela falta de autenticação e encadeamento, o invasor

torna-se capaz de executar ataques de retransmissão, injetando blocos capturados anteriormente, ou realizar a substituição e reordenação de blocos no meio do caminho, forçando o receptor a executar comandos espúrios sem que a manipulação seja detectada.

6.1 Demonstração Prática de Vulnerabilidade na UART

Para evidenciar o risco em um enlace serial sem proteção, simulou-se o envio de comandos de controle de atuadores com blocos repetidos sob a mesma chave:

Bloco	**Texto Claro (16 bytes)**	**Texto Cifrado em Modo ECB**
Bloco 1	`"LIGAR LED VERDE"`	`e0 c2 2a 6f 02 66 4f 48 7d c9 52 86 42 20 3d ec`
Bloco 2	`"DESLIGAR LED"`	`2b 4d 06 5e a6 54 78 b6 06 96 61 62 44 cd 50 70`
Bloco 3	`"LIGAR LED VERDE"`	`e0 c2 2a 6f 02 66 4f 48 7d c9 52 86 42 20 3d ec`

Como o **Bloco 3** repete o **Bloco 1**, o texto cifrado repete-se perfeitamente. Um atacante escutando a UART deduz instantaneamente quando o mesmo comando foi emitido e pode capturar o bloco e reinjetá-lo (*replay attack*) a qualquer momento para ligar o LED sem possuir a chave simétrica. A boa prática criptográfica recomenda modos com aleatoriedade e encadeamento:

- **CBC (*Cipher Block Chaining*)**: introduz um Vetor de Inicialização (*IV*) aleatório e exclusivo a cada mensagem, realizando uma operação XOR com o bloco anterior antes da cifragem;
- **GCM (*Galois/Counter Mode*)**: modo autenticado moderno (AEAD) que une a flexibilidade do modo contador (CTR) à autenticação por multiplicação no corpo de Galois $GF(2^{128})$, assegurando tanto a confidencialidade quanto a integridade estrita dos dados transmitidos.

7. REFERÊNCIAS

STALLINGS, William; BROWN, Lawrie. ***Computer Security: Principles and Practice***. 5. ed. Pearson, 2021.

NIST. ***FIPS PUB 197: Advanced Encryption Standard (AES)***. National Institute of Standards and Technology, 2001.

NIST. ***SP 800-38A: Recommendation for Block Cipher Modes of Operation: Methods and Techniques***. National Institute of Standards and Technology, 2001.